# Telegram Mini App Store

Buy consumer products from China, paid manually, managed from a web admin panel

Name: EasyBuy

# Part 1 - Product Architecture

## 1.1 Vision

A Telegram Mini App that lets customers browse and order consumer products sourced from Chinese marketplaces (Taobao, Pinduoduo, Poizon, 1688) without leaving Telegram. The store holds no inventory. Every order is purchased in China only after the customer's payment is confirmed, then inspected, consolidated and shipped. This must be explained clearly to customers on a dedicated **How it works** page so expectations about delivery time and the process are set before they pay.

## 1.2 Users and roles

| Role | Who | Main goals |
|---|---|---|
| **Customer** | Telegram user opening the Mini App | Find products, choose options, order, pay, track delivery, reorder |
| **Owner (admin)** | You | Full control: catalog, orders, payments, users, settings, admin accounts, analytics |
| **Manager (The seller)** | Staff you add | Process orders and payment reviews, edit products; no settings or admin accounts |
| | | |

## 1.3 Scope

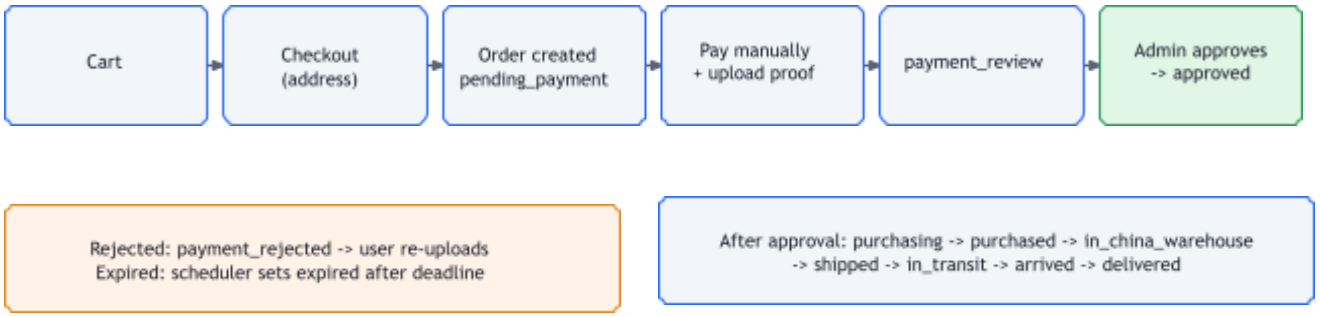
| In scope (v1) | Out of scope (later) |
|---|---|
| **Catalog with categories, search, variants (color, size, model)** | Online card payments / crypto gateways |
| **Likes (favorites), share links, buyer photos from Chinese platforms** | Automatic purchasing from Taobao/1688 APIs |
| **Cart, checkout, orders, order history, status timeline** | Multi-vendor marketplace and seller accounts |
| **Manual payment with screenshot upload and admin approval** | Coupons, loyalty points, referrals (see roadmap) |
| **Settings: language, addresses, phone, liked products** | Live chat inside the app (use Telegram link to support) |
| **Web admin panel (SQLAdmin) with dashboard and audit log** | Native iOS/Android apps |
| **Telegram notifications for every status change** | Parcel tracking API (tracking number is entered manually); admin roles, 2FA, banners, CSV export (later) |

## 1.4 Page map

| # | Page | Purpose | Key elements |
|---|---|---|---|
| 1 | Home | Discover products | Brand name, search bar, category chips, product grid, How it works entry |
| 1b | How it works | Set expectations | Steps: choose, pay, we buy in China, inspect, ship, receive. States we own no stock |
| 2 | Product detail | Decide and add to cart | Photo carousel, title, price, like, share, options, description, buyer photos, add to cart |
| 3 | Cart | Review selection | Items, quantity, remove, totals, checkout button |
| 4 | History | Past orders | Paginated list, status chips, reorder |

| # | Page | Purpose | Key elements |
|---|---|---|---|
| 5 | Order status | Track one order | Timeline, items, address, totals, tracking number, cancel when allowed |
| 6 | Payment | Pay manually | Amount, method details/QR, countdown, screenshot upload, review state, rejection reason |
| 7 | Settings | Account | Language, addresses, phone, liked products, support, terms |

## 1.5 End-to-end order flow




- The order is created on the **backend** at checkout with status **pending_payment**, prices are snapshotted, and a payment deadline is set (default 24 hours, configurable).
- The customer sees the Payment page with the exact amount and manual payment instructions, pays outside the app, and uploads a screenshot.
- Status becomes **payment_review**. The admin group in Telegram receives a message with Approve / Reject buttons, and the admin panel shows it in the review queue.
- Approve changes the order to **approved** and notifies the customer. Reject requires a reason and returns the order to the customer for re-upload.
- The admin then moves the order through purchasing, warehouse, shipping and delivery. Each change is recorded in the timeline and notifies the customer.

## 1.6 Product rules

- Prices are shown in one display currency chosen in settings (for example USD or UZS). Admin enters a base price per product or variant; the CNY source cost is visible only to admins.
- A fixed service fee percent and shipping fee are configurable in settings and shown separately at checkout. If shipping is calculated after purchase, show it as estimated and state that clearly.
- Customers can cancel only while the order is pending_payment or payment_rejected. After approval, cancellation is a manual admin decision (refund flow).
- Product options are defined by the admin per product: any option names (Color, Size, Model, Storage) and values. A variant is a valid combination with its own price and optional image.
- Buyer photos (from Taobao, Pinduoduo, Poizon, etc.) are shown after the description with a small platform badge. Admin uploads them; the app never scrapes them automatically.

## 1.7 Admin capabilities

| Area | What the admin can do |
|---|---|

| | |
|---|---|
| **Dashboard** | Orders today/week/month, revenue, average order value, pending payment reviews, status funnel, top products, 30-day chart |

| Area | What the admin can do |
|---|---|
| **Orders** | Search and filter, open details, change status with comment, enter tracking number, see timeline and proof, cancel/refund |
| **Payment review** | Queue of proofs, large image viewer, approve or reject with reason, duplicate-image warning |
| **Products** | Create, edit, hide, delete; media upload and ordering; options and variants editor; buyer photos; source link and CNY cost |
| **Categories** | Tree with ordering, icons, enable/disable |
| **Users** | Search, view profile and orders, block/unblock, edit phone and internal notes |
| **Content** | How it works page, terms, FAQ (banners later) |
| **Payment methods** | Title, instructions text, QR image, enable/disable, order |
| **Settings** | Brand name, currency, rates, fees, deadline hours, support link, notification chat id |
| **Security** | Single owner login, audit log (roles and 2FA later) |

## 2.1 Technology choices

| Layer | Choice | Why |
|---|---|---|
| **API** | FastAPI + Pydantic v2 | Async, typed, automatic OpenAPI docs, fast to build |
| **ORM / migrations** | SQLAlchemy 2.x async + Alembic | Mature, explicit, safe schema evolution |
| **Database** | PostgreSQL 16 | Transactions for orders and money, JSONB for i18n and snapshots, strong indexing |
| **Bot** | aiogram 3 (webhook) | Async, runs inside the same API process |
| **Jobs** | APScheduler inside the API + outbox table (no Redis) | Order expiry and reliable notifications with zero extra services; add Redis only when you run several API processes |
| **Mini App UI** | React + TypeScript + Vite + Tailwind | Fast, Telegram SDK support, large ecosystem |
| **Admin UI** | SQLAdmin (server-rendered, inside FastAPI) | CRUD for every table from the model definitions, plus two custom pages (payment review, dashboard); no second frontend to build or host |
| **Storage** | Cloudflare R2 (S3 API); local folder in development | Free 10 GB tier, no egress fees; private bucket for payment proofs |
| **Hosting** | One VPS, Docker Compose with 3 containers (caddy, api, db) | About USD 7 per month in total; easy to move to managed services later |
| **Observability** | structlog, Sentry free plan, UptimeRobot on /health | Find production problems at no cost |

**Why SQLAdmin for the MVP:** it gives you full CRUD for every table from the model definitions, so there is no second frontend to build, host or secure. The trade-off is a less polished interface than a custom React admin, and the payment review page and dashboard are small custom views. You can replace it later with a React admin without touching the database or business rules, because all rules live in service functions.

### 2.1.1 Architectural style

A **modular monolith**: one FastAPI codebase split into domain modules with clear boundaries, one process (scheduled jobs run inside it), and one database. This is the right size for a store at this stage; microservices add cost without benefit. Modules communicate through service functions, never by reaching into each other's tables.

## 2.2 Backend module layout

```
backend/
  app/
    main.py                  # FastAPI app, routers, bot webhook mount
    core/                    # config, security, db session, logging, errors, i18n
    modules/                 # init_data validation, JWT, admin login
      auth/                  # profile, addresses, phone, block
      users/
      catalog/               # categories, products, options, variants, media, search
      favorites/
      cart/
      orders/                # order creation, status machine, history
      payments/              # methods, proofs, review, expiry
      content/               # banners, pages, app settings
      admin/                 # SQLAdmin views, payment review page, dashboard, audit log
      notifications/         # message templates, outbox table, send via bot
    bot/                     # aiogram handlers, admin group callbacks
    jobs/                    # APScheduler: expire_orders, flush_outbox, backup, cleanup
    db/                      # models, alembic migrations, seed
  miniapp/                   # React Mini App
  infra/                     # docker-compose.yml, Caddyfile, backup script
```

## 2.3 Data model

All tables use bigint primary keys, created_at/updated_at timestamps, and soft state fields instead of hard deletes where history matters. Text shown to customers is stored as JSONB translations (for example {"en": "...", "ru": "..."}).

| Table | Key columns | Notes |
|---|---|---|
| **users** | telegram_id (unique), username, names, language, phone, is_blocked, last_seen_at | Created on first valid auth |
| **addresses** | user_id, recipient, phone, country, region, city, street, postal_code, lat/lng, is_default | One default per user |
| **categories** | parent_id, name_i18n, slug, icon, sort_order, is_active | Tree structure |
| **products** | category_id, title_i18n, description_i18n, base_price, currency, source_platform, source_url, source_price_cny, status, likes_count | source_* admin-only |
| **product_media** | product_id, url, kind (gallery or buyer_photo), source_platform, sort_order | Buyer photos show platform badge |
| **product_options** | product_id, name_i18n, sort_order | Color, Size, Model... |
| **product_option_values** | option_id, value_i18n, image_url, sort_order | Red, XL, Pro... |
| **product_variants** | product_id, sku, option_value_ids[], price, image_url, is_active | Price per combination |
| **favorites** | user_id, product_id | Composite primary key |

| Table | Key columns | Notes |
|---|---|---|
| **cart_items** | user_id, product_id, variant_id, quantity | Unique per user + variant |
| **orders** | number (unique), user_id, status, subtotal, shipping_fee, service_fee, total, address_snapshot, payment_deadline_at, tracking_number | NUMERIC(12,2) for money |
| **order_items** | order_id, product_id, variant_id, title_snapshot, options_snapshot, unit_price, quantity | Immutable snapshot |
| **order_status_histor y** | order_id, from_status, to_status, actor_type, actor_id, comment | Drives the timeline |
| **payment_methods** | title, details_text, qr_image_url, is_active, sort_order | Managed by admin |
| **payments** | order_id, method_id, amount, status, reviewed_by, reviewed_at, reject_reason | One active payment per order |
| **payment_proofs** | payment_id, file_key, sha256, uploaded_at | Multiple allowed after rejection |
| **admins** | email, password_hash, totp_secret, role, is_active | Argon2 hashes |
| **admin_audit_log** | admin_id, action, entity, entity_id, diff (JSONB) | Append-only |
| **banners / pages / app_settings** | image+link, slug+body_i18n, key+value JSONB | Content and configuration |

**Recommended indexes:** users(telegram_id); products(category_id, status, sort_order); GIN or trigram index on product titles for search; orders(user_id, created_at desc); orders(status, created_at); payment_proofs(sha256); order_status_history(order_id, created_at).

## 2.4 Authentication and authorization

### Customers (Mini App)

- The Mini App sends `Telegram.WebApp.initData` to `POST /api/v1/auth/telegram`.
- The server recomputes the hash: secret_key = HMAC_SHA256(key="WebAppData", msg=BOT_TOKEN); data_check_string = sorted key=value lines without hash; compare HMAC_SHA256(secret_key, data_check_string) to hash using constant-time comparison.
- Reject if auth_date is older than one hour. Upsert the user, return a short-lived JWT (15-60 min) plus a refresh path by repeating initData auth.
- Every customer endpoint derives user_id from the JWT. The client never sends its own user id.

### Admins (web panel)

- Email + strong password (Argon2), session cookie (HttpOnly, Secure, SameSite=Strict), login rate limiting.
- The MVP has one role (owner). Keep the role and TOTP columns so manager/support roles and 2FA can be added later without a migration. Every admin change writes to admin_audit_log.
- The admin lives under /admin, separate from the customer API; if you have a fixed IP, restrict /admin to it in Caddy.

### Telegram admin group buttons

- Callback data carries only an opaque id; the handler checks that the pressing Telegram user id is in the admin allow-list and then calls the same approval service as the web panel.

## 2.5 Storage and files

- Product media: public R2 bucket behind the Cloudflare CDN. The API resizes (for example 480 and 1200 px wide) and converts to WebP with Pillow at upload time; no separate worker.

- Payment proofs: **private** bucket. Admins view them through an authenticated endpoint (or a signed URL that expires in minutes). Customers can see only their own proof.
- Validate type by magic bytes, limit size (10 MB), strip EXIF, store sha256 to detect the same image reused on another order.

## 2.6 Security checklist

| Risk | Mitigation |
|---|---|
| **Forged Telegram identity** | Server-side initData HMAC validation, auth_date expiry |
| **Accessing other users' orders (IDOR)** | Always filter by authenticated user id; tests for cross-user access |
| **Price tampering** | Client sends only product/variant ids and quantities; server computes all prices |
| **Fake or reused payment screenshots** | Manual review, sha256 duplicate detection, amount shown in review screen, optional reference code per order |
| **Malicious uploads** | Type check by magic bytes, size limit, re-encode image, store outside web root |
| **Brute force on admin login** | Rate limiting, lockout, strong password, optional IP allowlist in Caddy, audit log (2FA later) |
| **Race conditions on status changes** | Single status service, row lock (SELECT FOR UPDATE), allowed-transition table |
| **Data loss** | Daily pg_dump to object storage, tested restore, bucket versioning |
| **Secret leakage** | Secrets only in environment variables, never in repo; rotate bot token if exposed |

# Part 3 - Product Documentation

## 3.1 Home

- **Header:** brand name (from settings), optional logo.
- **Search bar:** debounced (300 ms) search over product titles and tags; shows results grid, recent searches and empty state.
- **Category chips:** horizontally scrollable, All first. Selecting a chip filters the grid. Sub-categories appear in a second row.
- **Banners:** later (not in the MVP).
- **Product grid:** two columns, image, title (two lines), price, like heart. Infinite scroll, skeleton loaders.
- **How it works:** a clear entry card or header link opening the explanation page.

**How it works page (content outline)**

- 1. Choose products and options, add to cart.
- 2. Place your order and pay by the shown method, then upload your payment screenshot.
- 3. We confirm your payment. Only then we buy the items from the Chinese marketplace.
- 4. Items arrive at our China warehouse where we review and inspect them.
- 5. We ship to your country; you receive tracking updates here and in Telegram.
- Important: we do not hold stock. Everything is purchased after your order, so delivery takes longer than local shops. Show typical time ranges configured in settings.

## 3.2 Product detail

- Swipeable photo carousel with dots and fullscreen zoom.
- Title, price (updates when a variant is chosen), like button, share button (Telegram share link with startapp parameter).
- **Option selectors:** chips or swatches for each option the admin defined. Unavailable combinations are disabled.
- Description text, then **Photos from buyers** grid with platform badges (Taobao, Pinduoduo, Poizon, other). Tap to open fullscreen.
- Sticky bottom bar: quantity stepper and Add to cart. If options are not all chosen, the button prompts the user to choose them.
- Telegram integration: BackButton returns to the previous page, haptic feedback on add to cart.

## 3.3 Cart

- List with image, title, chosen options, unit price, quantity stepper, remove.
- Totals: subtotal, service fee, estimated shipping, total.
- Checkout button (Telegram MainButton) leads to address selection, then creates the order on the server.
- Edge cases: product hidden or price changed since adding -> show a notice and refresh totals; empty cart state links to Home.

## 3.4 History

- Paginated orders, newest first: number, date, thumbnail strip, total, status chip.

- Filters: All, Active, Delivered, Cancelled.
- Reorder button copies items back to the cart (using current prices).

## 3.5 Order status

- Vertical timeline built from order_status_history with timestamps and admin comments (for example the rejection reason).
- Order summary, address, tracking number with copy button once available.
- Actions by state: pending_payment -> Pay now / Cancel; payment_rejected -> Re-upload / Cancel; shipped and later -> tracking details.

| Status | Customer sees | Who sets it |
|---|---|---|
| **pending_payment** | Waiting for your payment | System |
| **payment_review** | Payment is being checked | System (on upload) |
| **payment_rejected** | Payment rejected + reason | Admin |
| **approved** | Payment confirmed | Admin |
| **purchasing** | We are buying your items in China | Admin |
| **purchased** | Items purchased | Admin |
| **in_china_warehouse** | Items received and inspected in China | Admin |
| **shipped / in_transit** | Parcel is on its way | Admin |
| **arrived** | Arrived in your country / pickup point | Admin |
| **delivered** | Delivered | Admin |
| **expired / cancelled / refunded** | Closed states | System / user / admin |

## 3.6 Payment

- Shows order number, **exact amount**, and the payment methods you configured (bank details, card number, QR). Copy buttons for each field.
- Countdown to the payment deadline. After expiry the order becomes expired and cannot be paid.
- Upload area: take photo or choose from gallery, preview, progress bar, retry on failure. Accepts jpg, png, webp, heic up to 10 MB.
- After upload the page shows **Waiting for review** with an estimated review time. The customer can leave the app; a Telegram message arrives on approval or rejection.
- On rejection show the admin's reason and enable re-upload. Optional: include a unique reference code the customer must write in the transfer comment, which makes review faster and fraud harder.

## 3.7 Settings

- **Profile:** name, Telegram username (read-only), language selection (en, ru, uz, zh ready).
- **Phone number:** via Telegram requestContact (verified) or manual entry stored with a note about verification.
- **Delivery addresses:** list, add, edit, delete, set default. Fields: recipient, phone, country, region, city, street, postal code, optional map pin.
- **Liked products:** grid of favorites with quick add to cart.
- Support (opens Telegram chat), Terms and Privacy, How it works.

## 3.8 Notifications

| Event | Customer message | Admin message |
|---|---|---|
| **Order created** | Order #123 created. Please pay within 24h. | - |
| **Proof uploaded** | We received your screenshot. | New proof for #123, buttons Approve / Reject |
| **Approved** | Payment confirmed. We will start purchasing. | Logged |
| **Rejected** | Payment rejected: reason. Please upload again. | Logged |
| **Each status change** | Status text + button to open the order | - |
| **Expired** | Order #123 expired because it was not paid. | - |

# Part 4 - Technical Documentation

## 4.1 Customer API (/api/v1)

| Method | Path | Description |
|---|---|---|
| **POST** | /auth/telegram | Validate initData, upsert user, return JWT |
| **GET** | /home | Banners, categories, first page of products, brand settings |
| **GET** | /categories | Category tree |
| **GET** | /products | Query params: q, category_id, sort, page, limit |
| **GET** | /products/{id} | Detail with media, options, variants, buyer photos, liked flag |
| **POST / DELETE** | /products/{id}/like | Like or unlike |
| **GET** | /me/likes | Liked products |
| **GET** | /cart | Cart with computed totals |
| **POST / PATCH / DELETE** | /cart/items[/{id}] | Add, change quantity, remove |
| **POST** | /orders | Create order from cart + address_id; returns order and payment info |
| **GET** | /orders | History, paginated, status filter |
| **GET** | /orders/{id} | Order detail with timeline |
| **POST** | /orders/{id}/cancel | Allowed only in pending_payment or payment_rejected |
| **GET** | /orders/{id}/payment | Amount, methods, deadline, current payment status |
| **POST** | /orders/{id}/payment-proof | multipart/form-data upload of screenshot |
| **GET / PATCH** | /me | Profile, language, phone |
| **GET / POST / PATCH / DELETE** | /me/addresses[/{id}] | Address CRUD, set default |
| **GET** | /pages/{slug} | how-it-works, terms, faq |
| **GET** | /settings/public | Brand, currency, fees, deadline hours, support link |

### Example: create order

```
POST /api/v1/ordersAut
 horization:Bearer<jwt
 >
{"address_id":12,"note":"Callbeforedelivery"}

201Created
 {
"id":981,"number":"A-2026-000981","status":"pending_payment",
"subtotal":"84.00","service_fee":"8.40","shipping_fee":"12.00","total":"10
   4.40","currency":"USD","payment_deadline_at":"2026-10-06T14:20:00Z",
   "items":[{"title":"Wirelessearbuds","options":{"Color":"Black"},"unit_price":"
     42.00","quantity":2}]
 }
```

## 4.2 Admin panel (SQLAdmin at /admin)

| Screen | Type | What it does |
|---|---|---|
| **Orders** | ModelView + actions | List and filter by status and date; detail with items, timeline and tracking number; Change status action with comment (calls change_status) |
| **Payment review** | Custom page | Queue of payments in review, large proof image, duplicate warning, Approve and Reject (reason required) |

| Screen | Type | What it does |
|---|---|---|
| **Products, options, variants, media** | ModelViews | CRUD with inline options and variants; image upload to R2; hide/show action; source link and CNY cost visible only here |
| **Categories** | ModelView | Tree CRUD with ordering and enable/disable |
| **Users** | ModelView | Search, block/unblock, edit phone and internal notes |
| **Payment methods, pages, settings** | ModelViews | Edit payment instructions, How it works text, fees and deadline without redeploying |
| **Dashboard** | Custom page | Orders today/week/month, revenue, pending reviews, status funnel, top products, 30-day chart |
| **Audit log** | Read-only ModelView | Who changed what and when |

**Service operations behind the admin screens.** The table below lists the operations the admin views call. They are plain service functions in the MVP; if you later build a React admin, expose the same functions under /admin/api.

| Area | Endpoints |
|---|---|
| **Auth** | POST /auth/login, POST /auth/logout, POST /auth/2fa/verify, GET /me |
| **Dashboard** | GET /dashboard/stats?range=7d\|30d\|90d (orders, revenue, funnel, top products, daily series) |
| **Orders** | GET /orders, GET /orders/{id}, POST /orders/{id}/status, PATCH /orders/{id} (tracking, notes), POST /orders/{id}/refund |
| **Payments** | GET /payments?status=review, POST /payments/{id}/approve, POST /payments/{id}/reject {reason}, GET /payments/{id}/proof-url |
| **Products** | CRUD /products, POST /products/{id}/media, PUT /products/{id}/media/order, CRUD options and variants, POST /products/bulk-status |
| **Categories** | CRUD /categories, PUT /categories/order |
| **Users** | GET /users, GET /users/{id}, PATCH /users/{id}, POST /users/{id}/block, POST /users/{id}/unblock |
| **Content** | CRUD /banners, PUT /pages/{slug}, CRUD /payment-methods, GET/PUT /settings |
| **Admins** | CRUD /admins (owner only), GET /audit-log |

## 4.3 Order status machine

```
pending_payment    ->payment_review|cancelle
 d|expiredpayment_review  -> approved | paymen
 t_rejectedpayment_rejected      ->payment_re
 view|cancelled
approved      ->purchasing|refunded
purchasing    -> purchased | refundedpurch
 ased  ->in_china_warehouse|refundedin_c
 hina_warehouse->shipped|refunded
shipped       ->in_transit
in_transit    ->arrived
arrived->delivered
(delivered,cancelled,expired,refundedareterminal)
```

All transitions go through `orders.service.change_status(order_id, to_status, actor, comment)`. It opens a transaction, locks the order row, checks the transition table, updates the order, writes order_status_history, writes the audit log for admin actors, and enqueues the Telegram notification. No other code may update orders.status directly.

## 4.4 Payment review flow (backend)

- POST /orders/{id}/payment-proof: verify ownership and status (pending_payment or payment_rejected), validate file, upload to private bucket, compute sha256, create payment_proofs row, set payment.status = review, call change_status(payment_review).

- Duplicate check: if sha256 exists on another order, mark the proof as flagged and show a warning in the admin queue (do not auto-reject).
- The service writes an outbox row; the scheduler sends the admin group message with the order number, amount, customer and Approve/Reject buttons.
- Approve: payment.status = approved, reviewed_by/at set, order -> approved. Reject: reason required, payment.status = rejected, order -> payment_rejected.
- Both actions are idempotent: pressing Approve twice must not duplicate history or notifications.

## 4.5 Scheduled jobs (APScheduler, inside the API process)

| Job | Schedule | Action |
|---|---|---|
| **expire_orders** | Every minute | pending_payment orders past payment_deadline_at -> expired via status service |
| **flush_outbox** | Every 5 seconds | Send pending Telegram messages from the outbox table with retries and backoff; log failures |
| **Image processing** | Inline on upload | Pillow resize, WebP conversion, EXIF strip (run in a thread, not a scheduled job) |
| **daily_backup** | Daily | pg_dump and upload to object storage, prune old backups |
| **cleanup** | Daily | Remove abandoned carts older than 60 days and orphaned uploads |

## 4.6 Telegram bot

- / s t a r t opens the Mini App with a Web App button; deep links ? s t a r t a p p = p r o d u c t _ 1 2 3 open a product directly.
- Webhook endpoint secured with a secret token header; mounted at /bot/webhook.
- Admin group: a notifications chat id is stored in app_settings; callback buttons Approve / Reject call the payment service. Only user ids in the admins allow-list can press them.
- Phone sharing uses the requestContact flow; the bot stores the number on the user.

## 4.7 Frontend notes (Mini App)

- Use the official Telegram Web App SDK: expand(), ready(), theme parameters, BackButton, MainButton, HapticFeedback, safe-area insets.
- State: TanStack Query for server data, a small store (Zustand) for UI state. Optimistic updates for likes and cart quantity.
- Routing with React Router; keep each of the 7 pages as a route; use code splitting.
- Images lazy loaded with responsive srcset; skeletons for every list; friendly error and offline states.
- i18n with JSON files per language; do not hard-code strings.

## 4.8 Testing strategy

| Level | Focus |
|---|---|
| **Unit** | initData validation, price calculation, status transitions, option/variant resolution |
| **API / integration** | Auth, cart to order creation, payment upload and approval, expiry job, cross-user access denied, admin permissions by role |
| **Frontend** | MVP: component tests for cart and payment upload and a manual full-flow test inside Telegram; Playwright end-to-end tests later |
| **Manual** | Real device test in Telegram iOS and Android, dark/light theme, slow network |

## 4.9 Configuration (environment variables)

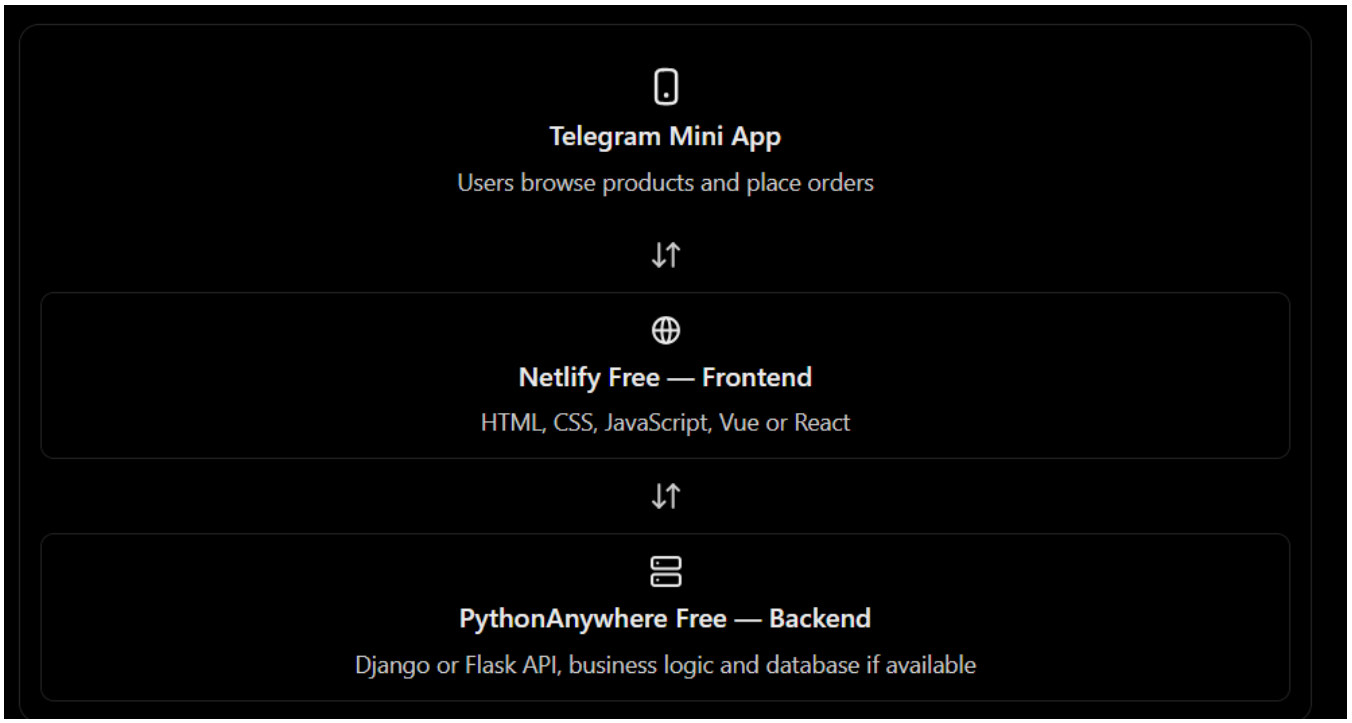
```
APP_ENV=productionDATABASE_URL=postg
 resql+asyncpg://user:pass@db:5432/shopBOT_
 TOKEN=...   #fromBotFather
BOT_WEBHOOK_SECRET=...   #randomstri
 ngWEBAPP_URL=https://shop.example.c
 omADMIN_TELEGRAM_IDS=111111,222222
 ADMIN_NOTIFY_CHAT_ID=-100123456789J
 WT_SECRET=...
ADMIN_EMAIL=...
ADMIN_PASSWORD_HASH=...  #argon2hashofthesingleownerpassw
 ordSTORAGE_BACKEND=r2   #or"local"indevelopment
S3_ENDPOINT=...     S3_BUCKET_PUBLIC=...      S3_BUCKET_PRIVATE=...      S3_A
 CCESS_KEY=...      S3_SECRET_KEY=...SENTRY_DSN=...
```

## 4.10 Delivery plan for the build

| Step | Work | Done when |
|---|---|---|
| 1 | Scaffold, docker-compose, models, migrations, seed | docker compose up works; seed data visible |
| 2 | Auth, users, addresses, settings endpoints | initData auth tests pass |
| 3 | Catalog, search, variants, likes | Product list and detail API complete |
| 4 | Cart, orders, status service, history | Order created with snapshot; transitions tested |
| 5 | Payments: methods, proof upload, review, expiry job | End-to-end approval works via API |
| 6 | Bot and notifications, admin group buttons | Every status change sends a message |
| 7 | Mini App frontend (7 pages) | Full flow works inside Telegram |
| 8 | SQLAdmin panel, payment review page, dashboard | Admin can run the whole order flow from the browser |
| 9 | Hardening, backups, deploy guide | Production checklist signed off |

## 4.11

- **Shipping fee model:** shipping fee until the home is written when the product is creating by the seller
- **Languages** Uzbek and Russian
- **Legal:** terms, refund policy and customs responsibility for your country; this document is not legal advice.


Telegram Mini App
Users browse products and place orders
Netlify Free — Frontend
HTML, CSS, JavaScript, Vue or React
PythonAnywhere Free — Backend
Django or Flask API, business logic and database if available